

PRACTICAL ACTIVITY 1 [CHALLENGE 1]: NSA Backdoor Write-up

NSA Backdoor

Hard

Cryptography

picoCTF 2022

backdoor

diffie_hellman

rsa

AUTHOR: JOSHUA INSCOE

Description

I heard someone has been sneakily installing backdoors in open-source implementations of Diffie-Hellman... I wonder who it could be... ;)

• gen.py

• output.txt

Hints ?

1

Look for Mr. Wong's whitepaper... His work has helped so many cats!

849 users solved

86%

Liked

🚩

picoCTF{FLAG}

Submit Flag

CHALLENGE TITLE: **NSA Backdoor**
(<https://play.picoctf.org/practice/challenge/283?category=2&difficulty=3&page=1&search=>)
PLATFORM: **picoCTF**
AUTHOR: **Joshua Inscoe**
CATEGORY: **Cryptography (Hard)**
TOOL USED: **Kali Linux**

Challenge Overview:

The NSA Backdoor challenge from picoCTF is a hard cryptography problem that explores backdoored implementations of Diffie-Hellman key exchange. Unlike typical RSA or standard DH challenges, this problem **involves a discrete logarithm scenario where the encryption raises a small base to the power of the secret flag instead of the usual modular exponentiation of plaintext.**

```
sys.stderr.write(f' {n}\n')
n = p * q
c = pow(3, FLAG, n)

print(f'n = {n.digits(16)}')
print(f'c = {c.digits(16)}')
```

—(jure@kali)~

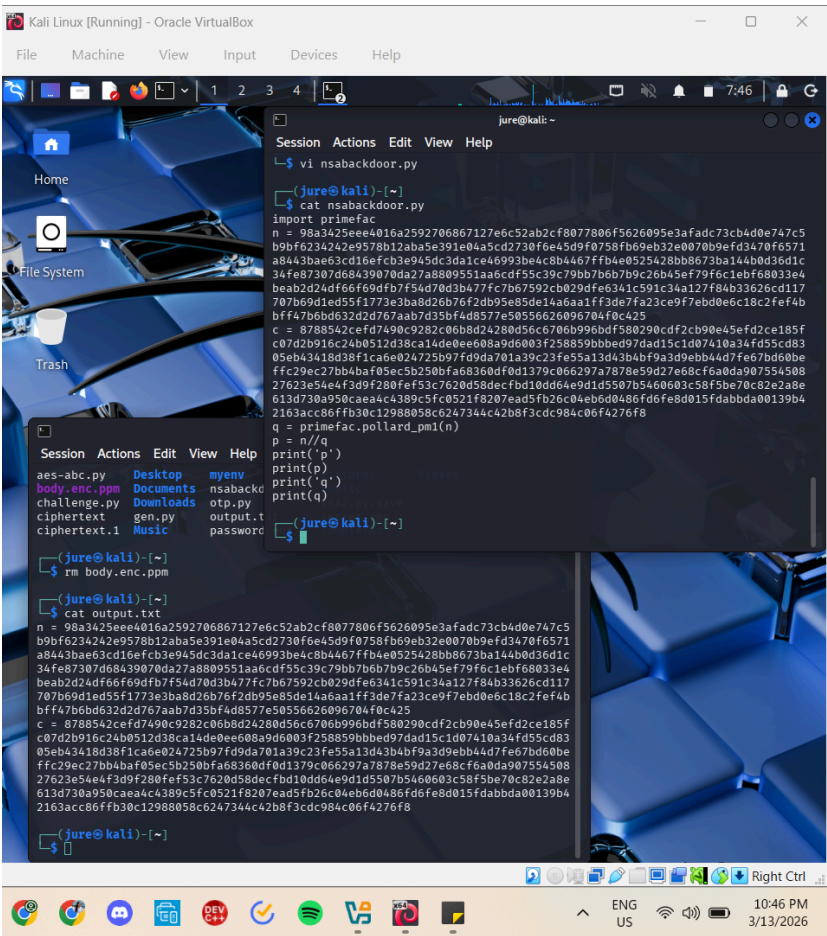
The challenge also references a white paper by David Wong, explaining how smooth-order primes and PolyCalman methods can be used to secretly embed vulnerabilities and solve discrete logarithm problems efficiently. This setup emphasizes how even mathematically sound systems can become insecure if backdoors or weak parameters are introduced.

PROCESS:

To solve the challenge, I first analyzed the provided code and parameters, identifying that the encryption relied on a composite modulus **n** and a ciphertext **c**, with the secret embedded as an exponent.

```
jure@kali: ~  
Session Actions Edit View Help  
jure@kali ~  
$ cat gen.py  
#!/usr/bin/python  
  
from binascii import hexlify  
from gmpy2 import *  
import math  
import os  
import sys  
  
if sys.version_info < (3, 9):  
    math.gcd = gcd  
    math.lcm = lcm  
  
_DEBUG = False  
  
FLAG = open('flag.txt').read().strip()  
FLAG = mpz(hexlify(FLAG.encode()), 16)  
SEED = mpz(hexlify(os.urandom(32)).decode(), 16)  
STATE = random_state(SEED)  
  
def get_prime(state, bits):  
    return next_prime(mpz_urandomb(state, bits) | (1 << (bits - 1)))  
  
def get_smooth_prime(state, bits, smoothness=16):  
    p = mpz(2)  
    p_factors = [p]  
    while p.bit_length() < bits - 2 * smoothness:
```

Following the guidance from Wong’s paper, I realized that factoring **n** using a smooth-order prime approach would allow me to compute the discrete logarithm using **PolyCalman**. I used a Python package called **primefact** to factor **n** into its prime components.



Sage Cell Server

sagecell.sagemath.org

About SageMathCell



Type some Sage code below and press Evaluate.

1

p=11763518096013972112731818983261071411459344063748615758282866116736427658121059934485733163

2

q=1638007298470299797112959410898000203002752116716613763962197756666888323537017528600576910

3

c= 0x8788542cefd7490c9282c06b8d24280d56c6706b996bdf580290cdf2cb90e45efd2ce185fc07d2b916c24b05

4

g=Mod(3,p)

5

m=discrete_log(c,g)

6

print(hex(m))

7

g2=Mod(3,q)

8

m2=discrete_log(c,g2)

9

print(hex(m2))

Language: Sage

Evaluate

Share

0x7069636f4354467b6233773472335f30665f63306d7030733174335f6d3064756c315f3939663338333377d
0x7069636f4354467b6233773472335f30665f63306d7030733174335f6d3064756c315f3939663338333377d

Help | Powered by SageMath

Sage Cell Server

Hex to String | Hex to ASCII Coi

rapidtables.com/convert/number/hex-to-ascii...

Hexadecimal

Text

Open File

Sample

Q

Paste hex code numbers or drop file

0x7069636f4354467b6233773472335f30665f63306d7030733174335f6d3064756c315f3939663338333377d
0x7069636f4354467b6233773472335f30665f63306d7030733174335f6d3064756c315f3939663338333377d

Character encoding

ASCII

= Convert

Reset

Swap

0000605Dg#7sG#50e630x00030C500GVÃ000c3003w0picoCTF{b3w4r3_of_c0mp0s1t3_m0d
u11_99f38837}

Copy

Save

Text to hex converter

ENG US

1:04 PM

3/13/2026

Then, leveraging SageMath, I set up the modular rings corresponding to the factors of n and applied the **discrete logarithm function** to recover the secret exponent. Finally, converting the result from hexadecimal to ASCII revealed the flag, successfully completing the challenge.

NSA Backdoor

Hard

Cryptography

picoCTF 2022

backdoor

diffie_hellman

rsa

AUTHOR: JOSHUA INSCOE

Description

I heard someone has been sneakily installing backdoors in open-source implementations of Diffie-Hellman... I wonder who it could be... ;)

• [gen.py](#)

• [output.txt](#)

Hints ?

1

Look for Mr. Wong's whitepaper... His work has helped so many cats!

849 users solved

86%

Liked

picoCTF{b3w4r3_of_c0mp0s1t3_m0dul1_99f38837}

Submit

Flag

Hurray! You solved this challenge.

Reflection:

This challenge highlighted the dangers of backdoored cryptography and the subtle ways a mathematically strong system like Diffie-Hellman can be compromised. By applying advanced techniques such as smooth-prime factorization, PolyCalman methods, and discrete logarithms, I gained hands-on experience in analyzing and exploiting cryptographic backdoors. The exercise reinforced that understanding the underlying math, implementation choices, and potential vulnerabilities is critical in cryptography, not just knowing the algorithms themselves.